

LAB SESSION 9

Programming Exercise:

Question 1: Create an abstract class Shape with a pure virtual function area(). Derive two classes Circle and Rectangle from Shape. Implement the area() function in both derived classes. Write a program to calculate the area of a circle and a rectangle.

```
PS C:\C++ PROGRAMS> cd "c:\C++ PROGRAMS\" ; if ($?) { g++ tut1.cpp -o  
tut1 } ; if ($?) { .\tut1 }  
Area of Circle: 78.5398  
Area of Rectangle: 24  
PS C:\C++ PROGRAMS>
```

```

#include <iostream>
#include <cmath> // for M_PI
using namespace std;

// Abstract class
class Shape {
public:
    // Pure virtual function
    virtual float area() = 0;
};

// Derived class: Circle
class Circle : public Shape {
private:
    float radius;

public:
    Circle(float r) {
        radius = r;
    }

    float area() override {
        return M_PI * radius * radius;
    }
};

// Derived class: Rectangle
class Rectangle : public Shape {
private:
    float length, width;

public:
    Rectangle(float l, float w) {
        length = l;
        width = w;
    }

    float area() override {
        return length * width;
    }
};

// Main function
int main() {
    // Create objects
    Circle c(5); // Radius = 5
    Rectangle r(4, 6); // Length = 4, Width = 6

    // Display areas
    cout << "Area of Circle: " << c.area() << endl;
    cout << "Area of Rectangle: " << r.area() <<
endl;
    return 0;
}

```

Question 2: Create an abstract class Employee with a pure virtual function calculateSalary(). Derive two classes PermanentEmployee and ContractEmployee from Employee. Implement the calculateSalary() function in both derived classes. Write a program to calculate the salary of a permanent employee and a contract employee.

```
PS C:\C++ PROGRAMS> cd "c:\C++ PROGRAMS\" ; if ($?) { g++ tut1.cpp -o  
tut1 } ; if ($?) { .\tut1 }  
Salary of Permanent Employee: 35000  
Salary of Contract Employee: 32000  
PS C:\C++ PROGRAMS>
```

```

#include <iostream>
using namespace std;

// Abstract class
class Employee {
public:
    // Pure virtual function
    virtual float calculateSalary() = 0;
};

// Derived class: PermanentEmployee
class PermanentEmployee : public Employee {
private:
    float basicSalary;
    float bonus;

public:
    PermanentEmployee(float basic, float b) {
        basicSalary = basic;
        bonus = b;
    }

    float calculateSalary() override {
        return basicSalary + bonus;
    }
};

// Derived class: ContractEmployee
class ContractEmployee : public Employee {
private:
    int hoursWorked;
    float hourlyRate;

public:
    ContractEmployee(int hours, float rate) {
        hoursWorked = hours;
        hourlyRate = rate;
    }

    float calculateSalary() override {
        return hoursWorked * hourlyRate;
    }
};

// Main function
int main() {
    // Create Permanent Employee: basic = 30000, bonus = 5000
    PermanentEmployee pEmp(30000, 5000);

    // Create Contract Employee: 160 hours, 200/hour
    ContractEmployee cEmp(160, 200);

    // Display salaries
    cout << "Salary of Permanent Employee: " << pEmp.calculateSalary() << endl;
    cout << "Salary of Contract Employee: " << cEmp.calculateSalary() << endl;

    return 0;
}

```

Question 3: Create an abstract class Vehicle with a pure virtual function fuelEfficiency(). Derive two classes Car and Truck from Vehicle. Implement the fuelEfficiency() function in both derived classes. Write a program to calculate the fuel efficiency of a car and a truck.

```
PS C:\C++ PROGRAMS> cd "c:\C++ PROGRAMS\" ; if ($?) { g++ tut1.cpp -o  
tut1 } ; if ($?) { .\tut1 }  
Car Fuel Efficiency: 20 km/l  
Truck Fuel Efficiency: 8 km/l  
PS C:\C++ PROGRAMS>
```

```

#include <iostream>
using namespace std;

// Abstract class
class Vehicle {
public:
    // Pure virtual function
    virtual float fuelEfficiency() = 0;
};

// Derived class: Car
class Car : public Vehicle {
private:
    float distance; // in kilometers
    float fuelUsed; // in liters

public:
    Car(float d, float f) {
        distance = d;
        fuelUsed = f;
    }

    float fuelEfficiency() override {
        return distance / fuelUsed; // km per liter
    }
};

// Derived class: Truck
class Truck : public Vehicle {
private:
    float distance; // in kilometers
    float fuelUsed; // in liters

public:
    Truck(float d, float f) {
        distance = d;
        fuelUsed = f;
    }

    float fuelEfficiency() override {
        return distance / fuelUsed; // km per liter
    }
};

// Main function
int main() {
    // Create a Car: 500 km driven, 25 liters used
    Car myCar(500, 25);

    // Create a Truck: 800 km driven, 100 liters used
    Truck myTruck(800, 100);

    // Display fuel efficiencies
    cout << "Car Fuel Efficiency: " << myCar.fuelEfficiency() << " km/l" << endl;
    cout << "Truck Fuel Efficiency: " << myTruck.fuelEfficiency() << " km/l" << endl;
    return 0;
}

```

Question 4: Create an abstract class Payment with a pure virtual function processPayment(). Derive two classes CreditCardPayment and PaypalPayment from Payment. Implement the processPayment() function in both derived classes. Write a program to process a payment through credit card and PayPal.

```
PS C:\C++ PROGRAMS> cd "c:\C++ PROGRAMS\" ; if ($?) { g++ tut1.cpp -o  
tut1 } ; if ($?) { .\tut1 }  
Processing credit card payment of $250.75  
Card Holder: John Doe  
Card Number: 1234-5678-9012-3456  
Payment successful via Credit Card.  
  
Processing PayPal payment of $99.99  
PayPal Account: john@example.com  
Payment successful via PayPal.  
  
PS C:\C++ PROGRAMS>
```

```

#include <iostream>
using namespace std;

// Abstract class
class Payment {
public:
    // Pure virtual function
    virtual void processPayment(float amount) = 0;
};

// Derived class: CreditCardPayment
class CreditCardPayment : public Payment {
private:
    string cardNumber;
    string cardHolder;

public:
    CreditCardPayment(string number, string holder) {
        cardNumber = number;
        cardHolder = holder;
    }

    void processPayment(float amount) override {
        cout << "Processing credit card payment of $" << amount << endl;
        cout << "Card Holder: " << cardHolder << endl;
        cout << "Card Number: " << cardNumber << endl;
        cout << "Payment successful via Credit Card.\n" << endl;
    }
};

// Derived class: PaypalPayment
class PaypalPayment : public Payment {
private:
    string email;

public:
    PaypalPayment(string mail) {
        email = mail;
    }

    void processPayment(float amount) override {
        cout << "Processing PayPal payment of $" << amount << endl;
        cout << "PayPal Account: " << email << endl;
        cout << "Payment successful via PayPal.\n" << endl;
    }
};

// Main function
int main() {
    // Create payment objects
    CreditCardPayment credit("1234-5678-9012-3456", "John Doe");
    PaypalPayment paypal("john@example.com");

    // Process payments
    credit.processPayment(250.75);    // Pay $250.75 via credit card
    paypal.processPayment(99.99);     // Pay $99.99 via PayPal

    return 0;
}

```


Question 5: Create an abstract class `Animal` with a pure virtual function `makeSound()`. Derive two classes `Dog` and `Cat` from `Animal`. Implement the `makeSound()` function in both derived classes. Write a program to simulate the sounds of a dog and a cat.

```
PS C:\C++ PROGRAMS> cd "c:\C++ PROGRAMS\" ; if ($?) { g++ tut1.cpp -o  
    tut1 } ; if ($?) { .\tut1 }  
Dog says: Woof! Woof!  
Cat says: Meow! Meow!  
PS C:\C++ PROGRAMS>
```



```
#include <iostream>
using namespace std;

// Abstract class
class Animal {
public:
    virtual void makeSound() = 0; // Pure virtual
};

// Derived class: Dog
class Dog : public Animal {
public:
    void makeSound() override {
        cout << "Dog says: Woof! Woof!" << endl;
    }
};

// Derived class: Cat
class Cat : public Animal {
public:
    void makeSound() override {
        cout << "Cat says: Meow! Meow!" << endl;
    }
};

// Main function
int main() {
    Dog myDog;
    Cat myCat;

    myDog.makeSound();
    myCat.makeSound();

    return 0;
}
```